

ROS2rapper インターフェース解説書

2026/APR/20

株式会社アックス

1. 概要	4
1.1. はじめに	4
1.2. ROS2とROS2rapper	5
1.3. サポートしている機能	6
1.3.1. トピックのパブリッシュ	6
1.3.2. トピックのサブスクライブ	6
1.3.3. Ethernet フレームの送信	6
1.3.4. Ethernet フレームの受信	6
1.4. 制限事項	7
1.5. ROS2・RTPS用語の対応	7
2. アーキテクチャ	8
3. ROS2rapper IPとユーザ・ロジックの接続	12
3.1. 概要	12
3.2. ROS2rapperのクロック、機能の有効化など	14
3.3. Ethernetモジュールとの接続	15
3.3.1. Ethernet MACとの接続	16
3.3.2. PHYの接続	17
3.4. パケット間隔に関するパラメータの設定	18
3.4.1. 合成時にパラメータで設定する場合	18
3.4.2. 動作時にユーザ・ロジックで管理する場合	18
3.5. ROS2rapperのポートの設定	20
3.6. アプリケーション・データ・メモリ(送信)の用意と接続	22
3.7. アプリケーション・データ・メモリ(受信)の用意と接続	23
3.8. IP受信ペイロード・メモリの用意と接続	24
3.9. Ethernet フレーム送信メモリの用意と接続	25
3.10. Ethernet フレーム受信メモリの用意と接続	26
3.11. ノード情報格納メモリの用意と接続	27
3.12. トピック情報格納メモリの用意と接続	28
4. ROS2通信	29
4.1. ROS2rapper各機能の有効化	29
4.2. パブリッシャの送信メッセージ(アプリケーション・データ・メモリ)の変更	30
4.3. サブスクライバの受信メッセージ(アプリケーション・データ・メモリ)の読み出し	32
5. Ethernet フレームの通信	35
5.1. Ethernet フレームの送信	35
5.2. Ethernet フレームの受信	38
6. 参考文献	39
7. 付録	40
7.1. ros2_etherモジュールのインターフェース仕様	40
7.2. ディレクトリ構成	55
7.3. RTPS通信で使用するUDPポート番号 一覧	56
8. 変更履歴	57

1. 概要

1.1. はじめに

“ROS2rapper”は、ROS2の通信のRTPSプロトコルなどのハードウェア実装であり、HLS C++とVerilogで記述されている。

本文書では、ROS2rapperを利用したシステムの設計を行うユーザーのための情報を提供する。

1.2. ROS2とROS2rapper

ROS2採用システムにおけるROS2rapperの位置づけを、Fig.1.1に示す。着色された部分が、ROS2rapperの各モジュールである。

ROS2rapperは、RTPS/UDP/IPプロトコルを実装している。それに加えて、ROS2rapperのIP及び実装サンプルには通信の物理層にEthernetを用いた実装も含んでいる。

なお、Ethernet以外の物理層も使用可能である。そのためには、物理層およびROS2rapperとのインターフェース部分を、別途実装すること。

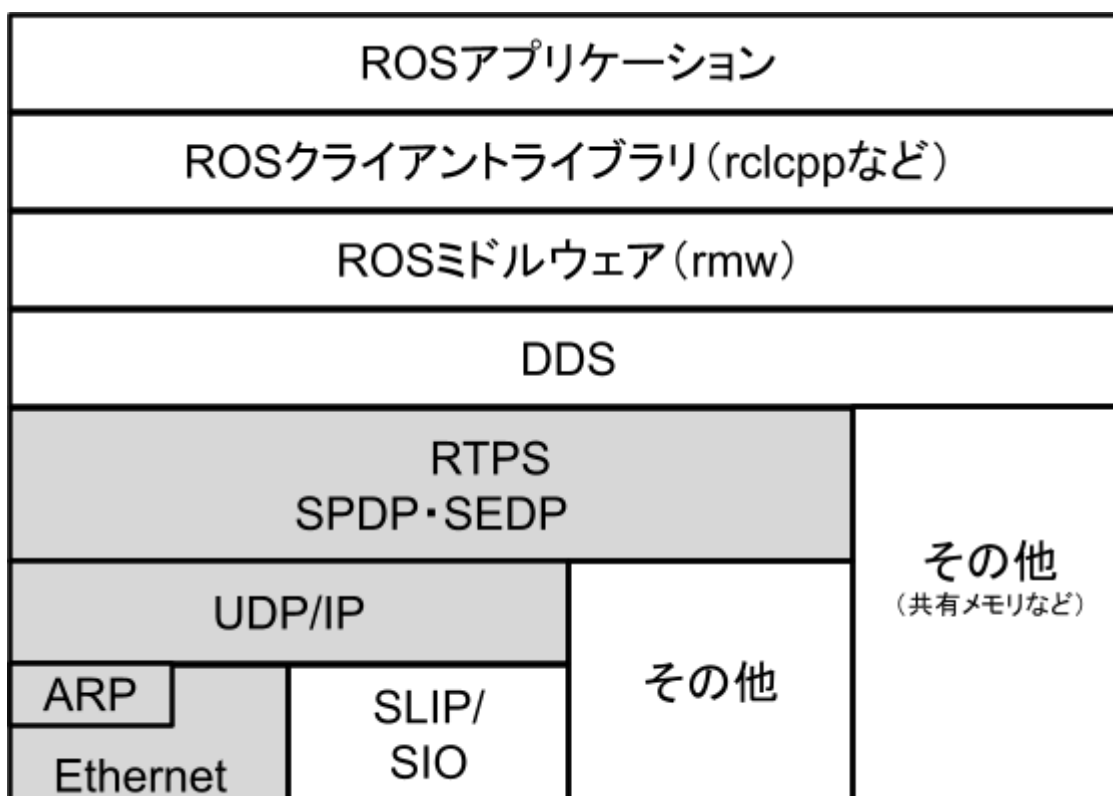


Fig.1.1 ROS2アーキテクチャにおけるROS2rapperの位置づけ

1.3. サポートしている機能

本IPが提供する機能をTable 1.1に示す。

本ROS2rapper IPは、下記の各機能を独立して個々に有効化/無効化できる。

- ROS2パブリッシュ機能
- ROS2サブスクライブ機能
- Ethernet

Table 1.1 サポート機能一覧

機能		内容	備考
プロトコル	RTPS SPDP SEDP	ROS2トピックのパブリッシュ	
		ROS2トピックのサブスクライブ	
通信物理層	Ethernet	Ethernet層 Ethernet フレームの送受信	Ethernet以外 の物理層に差し 替え可能
	ARP	物理アドレス解決	

1.3.1. トピックのパブリッシュ

ROS2ネットワークへトピックをパブリッシュする。トピックをサブスクライブしたすべてのノードに対し、一定周期でメッセージを送信する。メッセージの型は任意であるが、ユーザが適切にシリアライズする必要がある。

1.3.2. トピックのサブスクライブ

ROS2ネットワーク上のトピックをサブスクライブする。パブリッシャからメッセージが届くと、その内容をメモリ・インターフェースで外部メモリに書き出す。

その後、受信したトピック・受信したデータ長等のメタ情報を `ros2_sub_app_data_recvinfo` のFIFO で出力して、データの受信を通知する。

1.3.3. Ethernet フレームの送信

Ethernet フレームを送信する。

IPv4 のパケットを送信する際は、ARP モジュールによって、宛先の MAC アドレスが解決される。

1.3.4. Ethernet フレームの受信

Ethernet フレームを受信して出力する。

ただし、ARP と RTPS パケットは破棄される (詳細は後述)。

1.4. 制限事項

本ROS2rapperのIPには、以下の制限事項が存在する。

- ノード数は1個
- パブリッシュ/サブスクライブできるトピックは、それぞれ最大 4 個
- メッセージ・サイズや通信可能ノード数などの制限(Table A-3 定数一覧を参照)

※ 対向ノードの離脱検知が可能になった。(ver 2.0 より実装)

1.5. ROS2・RTPS用語の対応

本IPはRTPSの実装であるが、本書では主に、対応するROS2の用語で説明する。

なお、本文中やソースコード中に、一部RTPSの用語を使用している箇所がある。用語の読み替えのため、以下にその対照表を示す。

Table1.2 ROS2とRTPSの用語の対応

ROS2用語	RTPS用語
Node	Participant
Message	Application Data
Topic	Endpoint
Publisher	Writer
Subscriber	Reader

2. アーキテクチャ

ROS2rapperの内部の機能ブロック図を、Fig. 2.1に示す。

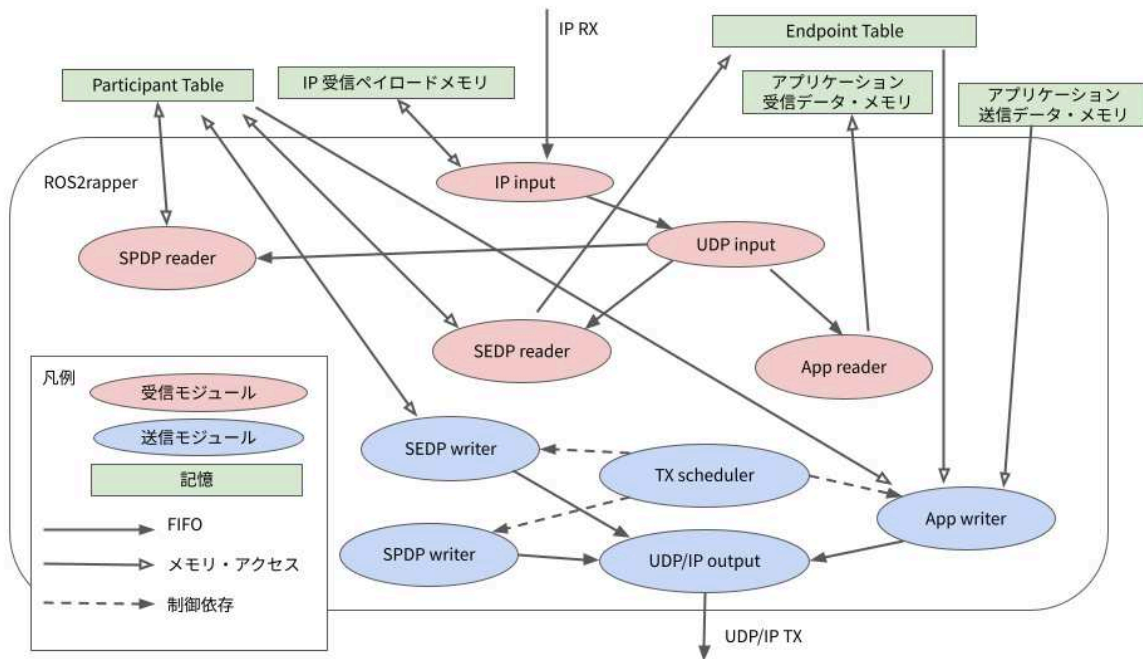


Fig. 2.1 ROS2rapper機能ブロック図

通信の物理層をEthernetとして、ROS2rapperを使用する時(ros2_ether)のモジュール構成図を、Fig. 2.2に示す。

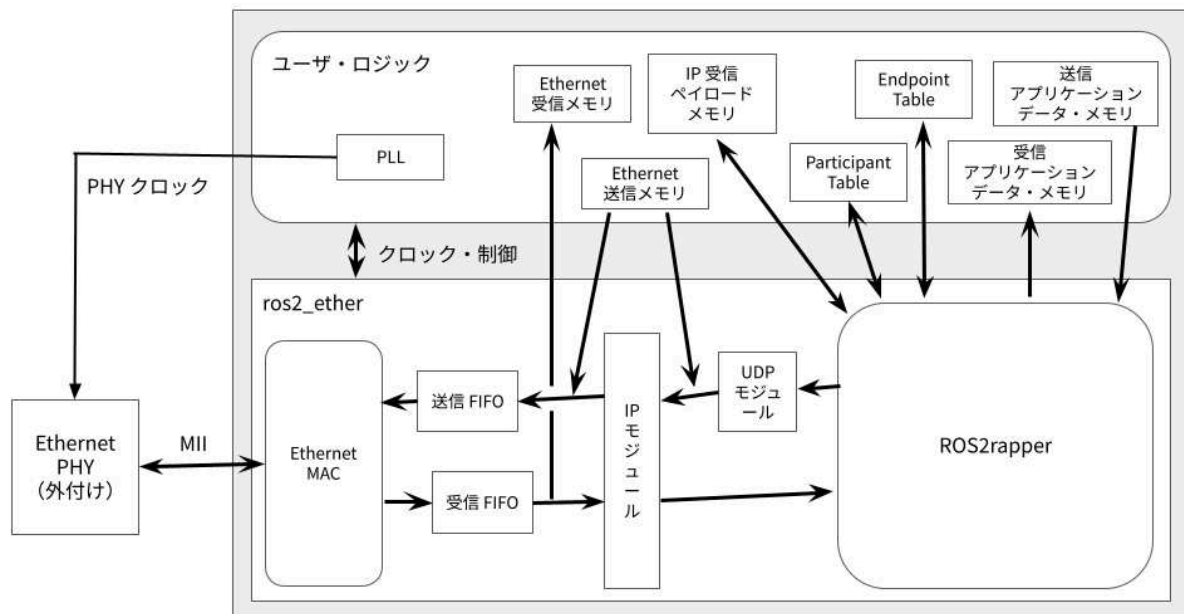


Fig.2.2 ROS2rapperのEthernet使用時のモジュール構成

以下に、主なモジュールの概要を記述する。

- ros2_ether
 - ros2_etherは、下記のサブ・モジュールから成る。
 - ROS2rapper
 - RTPS/UDP/IP通信 各層の実装
 - UDP モジュール
 - 送信パケットのチェックサム計算を行う。
 - IP モジュール
 - IP チェックサム計算・ARP 処理を行う。
 - Ethernet MAC
 - EthernetのMAC層
 - 送信FIFO
 - Ethernet MACへのEthernetフレームをバッファリングするためのFIFO
 - 受信FIFO
 - Ethernet MACからのEthernetフレームをバッファリングするためのFIFO
- ROS2rapper 以外のサブ・モジュールには、verilog-ethernet (参考文献 [5]) の実装を一部改変して使用した。
- アプリケーション・データ・メモリ(送信)
ROS2パブリッシャとして送信するデータを保持するメモリ。ユーザ・ロジックで用意する。
- アプリケーション・データ・メモリ(受信)
ROS2サブスクリバとして受信したデータを保持するメモリ。ROS2rapper IPから書き込まれる。ユーザ・ロジックで用意する。
- Ethernet フレーム送信メモリ
Ethernet フレームとして送信するデータを保持するメモリ。ユーザ・ロジックで用意する。
- Ethernet フレーム受信メモリ
受信した Ethernet フレームを保持するメモリ。ROS2rapper IPから書き込まれる。ユーザ・ロジックで用意する。
- IP受信ペイロード・メモリ
RTPS 通信で使用する、受信したIPパケットのフラグメントを格納して、全フラグメントの到着を待ち合わせるためのIP受信用のメモリ。
ユーザは、このメモリを用意し、接続しなければならない。
 - ユーザが用意すべきメモリの大きさは、バイト単位で、
$$\text{MAX_PENDINGS} \times \text{IP_MAX_PAYLOAD_LEN} \times \text{MAX_IP_FRAGMENTS}$$
で計算できる。
定数のそれぞれの意味は次の通り。
 - MAX_PENDINGS、IP_MAX_PAYLOAD_LEN、MAX_IP_FRAGMENTSの積で計算される。定数のそれぞれの意味は次の通り。
 - MAX_PENDINGS: 同時にIPフラグメント組み立て待ちを行えるIPデータグラムの数
 - IP_MAX_PAYLOAD_LEN: 最大IP通信ペイロードサイズ
 - MAX_IP_FRAGMENTS: 最大IPフラグメント数
 - デフォルト値はTable A-2を参照のこと。
 - フラグメント化された RTPS パケットの受信を行わない場合は、このIP受信ペイロード・メモリは不要

- ノード情報格納メモリ (Participant Table)
RTPS 通信で使用する、通信相手のノードの情報を格納するメモリ。
 - RTPS 通信を行わない場合は不要
- トピック情報格納メモリ (Endpoint Table)
RTPS 通信で使用する、通信相手のトピック毎の情報を格納するメモリ。
 - RTPS 通信を行わない場合は不要
- ユーザ・ロジック
ユーザの実現した機能を実装した論理であり、各モジュールの初期化や送受信メッセージの作成、送受信を行う。
 - アプリケーション・データ・メモリ(送信)
 - パブリッシュするメッセージの記憶域
 - アプリケーション・データ・メモリ(受信)
 - サブスクライブしたトピックより受信したメッセージの記憶域

3. ROS2rapper IPとユーザ・ロジックの接続

3.1. 概要

ここでは、ユーザ・ロジックを作成する開発者向けに、本ROS2rapper IPとユーザの論理(ユーザ・ロジック)を接続するための開発について説明する。

基本事項

- ROS2rapper IPには外部からクロックを与えること。
- ROS2rapper IPの入力ポートは信号線であるから、適宜ユーザ・ロジックなどにレジスタ(D Flip Flop)を設けるなどして値を保持して、ポートへの信号を安定供給しなければならない。
- ポートに与える信号の変化は、ROS2rapperに与えるクロックに同期していること。
- 複数ビットからなるポートや信号線は、LSB 0(最下位ビットが、ビット番号0)である。

動作確認を行った環境

- 高位合成ツール
 - Vitis HLS 2023.2 (Xilinx)
 - CyberWorkBench 11.0 Enterprise for Linux 64-bit (NEC)
- RTL 合成ツール
 - Vivado 2023.2 (Xilinx)
- FPGA 基板
 - Arty A7-100T (Digilent)
 - Artix Ultrascale+ AU25P (Xilinx)

開発の概略手順

1. 以下の通り Verilog マクロを定義する。
 - Vitis HLS で高位合成する場合は ROS2RAPPER_HLS_VITIS、CyberWorkBench で高位合成する場合は ROS2RAPPER_HLS_CWB を定義する。
 - Vivado で RTL 合成を行う場合は TARGET_XILINX を定義する。また、Vivado を使用する場合は、src/ether/include/ros2_ether_config.vh のコメントにしたがって、XILINX_CLKIN_STYLE_{BUFG,BUFR,BUFIO,BUFIO2} のいずれかを定義すること。
2. 次のVerilogヘッダファイルをインクルードする。
 - ros2rapper/src/ros2rapper/include/ros2_config.vh
 - ros2rapper/src/ether/include/ros2_ether_config.vh
3. ros2_ether.vにあるros2_etherをインスタンス化する。
 - その定義はros2rapper/src/ether/rtl/ros2_ether.vにある
 - ros2_etherモジュールのポートとの接続、IP受信ペイロード・メモリとの接続を行う

次節以降ではFPGA(ASIC)へROS2rapperを接続する上での要点と、ユーザ・ロジック作成上の留意点やヒントとなる情報を記述する。

実装例として、examplesディレクトリ中の以下のコードを参照のこと。

- topic-pub-sub/top.v : トピックのパブリッシュ/サブスクライブの実装例
- raw-eth-tx-rx/top.v : Ethernet フレーム送受信の実装例
- multi-topic-publication/top.v : 複数のトピックをパブリッシュする実装例
- multi-topic-subscription/top.v : 複数のトピックをサブスクライブする実装例
- test-1KB-pub-sub/rtl/ : ROS2, Ethernet フレーム送受信の最大長での使用例
- topic-pub-sub-au25p/top.v : Artix Ultrascale+ AU25P での実装例

付録には以下の技術情報や、参考情報へのリンクを記述する。

- ROS2rapperのIPで、Fig2.2に示した ros2_etherモジュールのポート仕様
- 同ポート使用一覧表中の定数と設定の注意事項
- 参考文献、特に以下の技術情報へのリンク
 - ノード名・トピック名・トピック型名の制限: ROS2やRTPSの仕様書
 - アプリケーションデータのシリアルライズ仕様: OMGの仕様書

3.2. ROS2rapperのクロック、機能の有効化など

3.2.1 ROS2rapperのクロック

ROS2rapper IPへのクロック clkは、外部から供給する。

3.2.2 ROS2rapperのリセット

ROS2rapper IPのリセットはrst_nを0にすることで行われる。

rst_nは非同期でよい。

3.2.2 ROS2rapper各機能の有効化

先述のとおり、本ROS2rapper IPは、下記の各機能を独立して個々に有効化/無効化できる。

- ROS2パブリッシュ機能, ros2sub_enにより制御
- ROS2サブスクライブ機能, ros2pub_enにより制御
- Ethernet MAC, ether_enにより制御

パブリッシュとサブスクライブは独立して個々に、または同時に使用できる

ROS2rapperの各機能を有効化するには、順序がある。

後述する各機能のポートの設定を行った上で、

物理通信手段(本実装サンプルではEther MAC)を有効化した後に、

ROS2のパブリッシュ、あるいはサブスクライブの機能の有効化を行わなければならない。

Table 3.1. ROS2rapperシステム・ポートの設定例

ROS2rapperクロック/リセット			
ポート名	方向 (ROS2rapper から見て)	ビット幅	説明
clk	input	1	クロック(50MHz以上)
rst_n	input	1	リセット(0でアサート)
ros2pub_en	input	4	4トピックそれぞれのパブリッシュを有効にする(1でアサート)
ros2sub_en	input	4	4トピックそれぞれのサブスクライブを有効にする(1でアサート)
ether_en	input	1	Ethernet MACを有効にする(1でアサート)。ether_enを一度アサートした後、デアサート(0)してはならない

3.3. Ethernetモジュールとの接続

ethernetモジュールとの接続は下記の2つがある

- Ethernet MAC
- PHY

3.3.1. Ethernet MACとの接続

下記のポートはEthernet MACを制御する。

ユーザ・ロジックを各ポートに接続して、使用するネットワーク環境に応じた設定値を与える。

Ethernet MACを有効化するには、ポートのether_enをアサート(1)する。

ether_enは一度アサートしたら、デアサート(0)してはならない。

- Ethernet・IPプロトコル
ether_en
mac_addr
ip_addr
gateway_ip_addr
subnet_mask
- ARP
arp_req_retry_count
arp_req_retry_interval
arp_req_timeout

実装サンプルではROS2rapperは通信にEthernet MACを使用するため、Ethernet MACに適切な値を与えている。

設定例をTable 3.2.に記述する。

Table 3.2. Ethernet MAC関連のポートの設定例

ポート名	設定値の例	説明
ether_en	1	Ethernet MAC有効(すべての設定値が与えられてからアサート(1)すること。アサート後にデアサート(0)してはならない)
mac_addr	48'h00_00_00_00_00_02	6バイト
ip_addr	{8'd100, 8'd1, 8'd168, 8'd192}	192.168.1.100の場合
gateway_ip_addr	{8'd1, 8'd1, 8'd168, 8'd192}	192.168.1.1の場合
subnet_mask	{8'd0, 8'd255, 8'd255, 8'd255}	255.255.255.0の場合
arp_req_retry_count	4	ARPの試行の最大回数
arp_req_retry_interval	(125000000*2)	単位:クロック数
arp_req_timeout	(125000000*30)	単位:クロック数

3.3.2. PHYの接続

クロック、リセット、Ethernet PHYの各端子とは以下のポートで接続する。
PHYのリファレンス・クロックは、ユーザ・ロジックで25MHzを生成して供給すること。

Table 3.3. クロック、リセット、Ethernet PHYのポート

Ethernet MII			
ポート名	方向 (ROS2rapper から見て)	ビット幅	説明
phy_rx_clk	input	1	Ethernet MII 受信クロック
phy_rxd	input	4	Ethernet MII 受信データ
phy_rx_dv	input	1	Ethernet MII受信データ有効
phy_rx_er	input	1	Ethernet MII 受信エラー
phy_tx_clk	input	1	Ethernet MII 送信クロック
phy_txd	output	4	Ethernet MII 送信データ
phy_tx_en	output	1	Ethernet MII 送信イネーブル
phy_rst_n	output	1	Ethernet MII リセット

3.4. パケット間隔に関するパラメータの設定

ROS2rapper が送信するパケットの間隔は、パラメータで合成時に設定する方法と、動作時にユーザ・ロジックで管理する方法がある。

どちらを選択するかは、合成時にパラメータ SET_TX_PERIOD_BY_PARAMETER で設定する。このパラメータを 1 にすると、パケット間隔を合成時にパラメータで設定する。このパラメータを 0 にすると、回路の動作時にユーザ・ロジックで管理する。

3.4.1. 合成時にパラメータで設定する場合

Table A-2 のパラメータ一覧を参照のこと。

こちらの場合は、回路合成時にクロック周波数を決定する必要がある。また、こちらの場合、Table 3.4 のポートは使用しない。

RTPSの制御パケット (SPDP, SEDP のパケット) の送信間隔は、通信相手検出までの時間やネットワーク帯域に応じて変更すること。

SPDP パケットの送信間隔は、後述の `ros2_participant_lease_duration_{seconds, fraction}` の設定値より、充分短くなければならない。詳細は、参考文献 [3] の “8.5.3 The Simple Participant Discovery Protocol” を参照のこと。

アプリケーション・データのパケットの送信間隔は、ROS2rapperのトピックをサブスクライブしたノードにメッセージが届く間隔である。アプリケーションに応じて変更すること。

3.4.2. 動作時にユーザ・ロジックで管理する場合

こちらの場合は、ROS2rapper の動作クロック周波数を回路の合成後に変更することが可能である。また、こちらの場合、Table A-2 のパラメータは SET_TX_PERIOD_BY_PARAMETER 以外使用しない。

以下のポートを用いて制御を行う：

Table 3.4. パケット間隔に関連するポート

ポート名	方向 (ROS2rapper から見て)	ビット幅	説明
ros2_timestamp_increment	input	32	クロック周期 [2^{32} sec]
ros2_cnt_interval_elapsed	input	1	パケット送信開始
ros2_cnt_spdp_wr_elapsed	input	1	SPDP Writer のパケット送信開始
ros2_cnt_sedp_pub_wr_elapsed	input	1	SEDP Publications Writer のパケット送信開始
ros2_cnt_sedp_sub_wr_elapsed	input	1	SEDP Subscriptions Writer のパケット送信開始
ros2_cnt_sedp_pub_hb_elapsed	input	1	SEDP Publications Writer の HEARTBEAT パケット送信開始

ros2_cnt_sedp_sub_hb_elapsed	input	1	SEDP Subscriptions Writer の HEARTBEAT パケット送信開始
ros2_cnt_sedp_pub_an_elapsed	input	1	SEDP Publications Writer の ACKNACK パケット送信開始
ros2_cnt_sedp_sub_an_elapsed	input	1	SEDP Subscriptions Writer の ACKNACK パケット送信開始
ros2_cnt_app_elapsed	input	1	アプリケーション・データの送信開始
ros2_cnt_interval_set	output	1	パケット送信完了
ros2_cnt_spdp_wr_set	output	1	SPDP Writer のパケット送信完了
ros2_cnt_sedp_pub_wr_set	output	1	SEDP Publications Writer のパケット送信完了
ros2_cnt_sedp_sub_wr_set	output	1	SEDP Subscriptions Writer のパケット送信完了
ros2_cnt_sedp_pub_hb_set	output	1	SEDP Publications Writer の HEARTBEAT パケット送信完了
ros2_cnt_sedp_sub_hb_set	output	1	SEDP Subscriptions Writer の HEARTBEAT パケット送信完了
ros2_cnt_sedp_pub_an_set	output	1	SEDP Publications Writer の ACKNACK パケット送信完了
ros2_cnt_sedp_sub_an_set	output	1	SEDP Subscriptions Writer の ACKNACK パケット送信完了
ros2_cnt_app_set	output	1	アプリケーション・データの送信完了

- ros2_cnt_interval_{elapsed,set} は、ROS2rapper 全体のパケット送信を制御する。
- その他 (ros2_cnt_interval_* 以外) の ros2_cnt_*_{elapsed,set} は、それぞれの種類のパケット送信を制御する。
- 一定の間隔で ros2_cnt_*_elapsed をアサート (1 と) することで、パケットを周期的に送信する、という使用方法を想定している。
- ros2_cnt_*_elapsed をアサート (1 と) した後、対応する ros2_cnt_*_set が 1 になるまで、ros2_cnt_*_elapsed は 1 のまま保持しなければならない。
- 例えば、SPDP パケットを送信する場合は、以下の手順で送信する (手順 a, b の順序は逆でもよい):
 - a. ros2_cnt_spdp_wr_elapsed をアサート (1 と) する。
 - b. ros2_cnt_interval_elapsed をアサート (1 と) する。(他に送信すべき RTPS パケットがない場合、ここで SPDP パケットの送信処理が開始される)
 - c. 何らかの RTPS パケットの送信処理が完了すると、ros2_cnt_interval_set が 1 となる。このときに ros2_cnt_interval_elapsed をデアサート (0 と) する。
(その後、一定時間が経過したら再び ros2_cnt_interval_elapsed をアサートする)
 - d. SPDP パケットの送信処理が完了すると、ros2_cnt_spdp_wr_set が 1 となる。このときに、ros2_cnt_spdp_wr_elapsed を 0 とする。

3.5. ROS2rapperのポートの設定

ROS2rapperの制御は、ユーザ・ロジックを、下に示すポートに接続し、適切な値を与えることで行う。RTPS のベンダー ID については、参考文献 [4] を参照のこと。また、`ros2_participant_lease_duration_{seconds, fraction}` の設定値は、SPDP パケットの送信間隔より充分長くなければならない。詳細は、参考文献 [3] の “8.5.3 The Simple Participant Discovery Protocol” を参照のこと。

Table 3.5. ROS2rapperのポートの設定例

ポート名	設定値の例	説明
<code>ros2_vendor_id</code>	16'h00_00	RTPS のベンダー ID
<code>ros2_node_udp_port</code>	16'd52000	ROS2rapper が RTPS 通信で使用する、送信元の UDP ポート番号
<code>ros2_port_num_seed</code>	16'd7400	計算式: (ROS2のDomain ID)*250+7400
<code>ros2_guid_prefix</code>	96'h00_00_00_00_00_00_09_de_ad_37_0f_01	ROS2rapper の GUID prefix
<code>ros2_participant_lease_duration_seconds</code>	32'd20	RTPS の participant lease duration の整数部 [sec]
<code>ros2_participant_lease_duration_fraction</code>	32'd0	RTPS の participant lease duration の小数部 [2^{-32} sec]
<code>ros2_node_name_len</code>	19	単位: バイト数 ('\\0'を含める)
<code>ros2_pub_topic_name_len_0</code>	11	単位: バイト数 ('\\0'を含める)
<code>ros2_pub_topic_name_len_1</code>		同上
<code>ros2_pub_topic_name_len_2</code>		同上
<code>ros2_pub_topic_name_len_3</code>		同上
<code>ros2_sub_topic_name_len_0</code>		同上
<code>ros2_sub_topic_name_len_1</code>		同上
<code>ros2_sub_topic_name_len_2</code>		同上
<code>ros2_sub_topic_name_len_3</code>		同上
<code>ros2_pub_topic_type_name_len_0</code>	29	単位: バイト数 ('\\0'を含める)
<code>ros2_pub_topic_type_name_len_1</code>		同上
<code>ros2_pub_topic_type_name_len_2</code>		同上
<code>ros2_pub_topic_type_name_len_3</code>		同上
<code>ros2_sub_topic_type_name_len_0</code>		同上
<code>ros2_sub_topic_type_name_len_1</code>		同上

ros2_sub_topic_type_name_len_2		同上
ros2_sub_topic_type_name_len_3		同上
ros2_pub_app_data_len_0	25	単位:バイト数
ros2_pub_app_data_len_1		同上
ros2_pub_app_data_len_2		同上
ros2_pub_app_data_len_3		同上
ros2_node_name	"elpmaxe_reppar2sor"	
ros2_pub_topic_name_0	"rettahc/tr"	
ros2_pub_topic_name_1		
ros2_pub_topic_name_2		
ros2_pub_topic_name_3		
ros2_sub_topic_name_0		
ros2_sub_topic_name_1		
ros2_sub_topic_name_2		
ros2_sub_topic_name_3		
ros2_pub_topic_type_name_0	"_gnirtS::_sdd::gsm::sgsm_dts"	
ros2_pub_topic_type_name_1		
ros2_pub_topic_type_name_2		
ros2_pub_topic_type_name_3		
ros2_sub_topic_type_name_0		
ros2_sub_topic_type_name_1		
ros2_sub_topic_type_name_2		
ros2_sub_topic_type_name_3		

3.6. アプリケーション・データ・メモリ(送信)の用意と接続

アプリケーション・データ・メモリは、ROS2rapperがパブリッシュしたトピックとして送信するデータを保持するメモリである。

ROS2rapper IPIは、パブリッシャ動作時に、外部へ送出するメッセージをアプリケーション・データ・メモリから得る。

アプリケーション・データ・メモリは、ユーザ・ロジックとして、メモリまたはD-FFで実現して用意し接続する。

アプリケーション・データ・メモリは、ポート`ros2_pub_app_data_{0,1,2,3}_{addr,ce,rdata}`と接続する。

なお、アプリケーション・データ・メモリの変更には、後述の手順が必要である。

ROS2rapper に接続するアプリケーション・データ・メモリ (送信) のインターフェースは、以下の仕様とすること。

- ポート数: 1R
- データ幅: 32ビット
- アドレス幅: $\lceil \log_2(\text{ROS2_MAX_APP_DATA_LEN}/4) \rceil$ ビット
- ワード数: $\text{ROS2_MAX_APP_DATA_LEN}/4$
- クロックに同期して読み出しを行う
- 以下のポートを持つ
 - ce: チップイネーブル(1でアサート)
 - addr: アドレス
 - rdata: 読み出しデータ(出力)
- ceがアサート(1)されているとき、アドレスaddrのデータをrdataに出力する。

3.7. アプリケーション・データ・メモリ(受信)の用意と接続

アプリケーション・データ・メモリは、ROS2rapperがサブスクライブしたトピックから受信したメッセージを保持するRAMである。

ROS2rapper IPIは、サブスクライバ動作時に、外部から受信したメッセージをアプリケーション・データ・メモリに格納する。

先述のとおり、アプリケーション・データ・メモリは、ユーザ・ロジックとして、メモリまたはD-FFで実現して用意し接続する。

アプリケーション・データ・メモリは、ポート`ros2_sub_app_data_{0,1,2,3}_{addr, ce, we, wdata}`と接続する。

なお、アプリケーション・データ・メモリの読み出しには、後述の手順が必要である。

ROS2rapper に接続するアプリケーション・データ・メモリ (受信) のインターフェースは、以下の仕様とすること。

- ポート数: 1W
- データ幅: 8ビット
- アドレス幅: $\lceil \log_2(\text{ROS2_MAX_APP_DATA_LEN}) \rceil$ ビット
- ワード数: `ROS2_MAX_APP_DATA_LEN`
- クロックに同期して書き込みを行う
- 以下のポートを持つ
 - `ce`: チップイネーブル(1でアサート)
 - `we`: ライト・イネーブル(1でアサート)
 - `addr`: アドレス
 - `wdata`: 書き込みデータ
- `ce`がアサート(1)されていて、かつ`we`がアサート(1)されているとき、アドレス`addr`に`wdata`の値を書き込む。

3.8. IP受信ペイロード・メモリの用意と接続

IP受信ペイロード・メモリは、ROS2機能有効時に受信したIPフラグメント組み立てに使用する。
IP受信ペイロード・メモリのサイズは、2960バイト程度が望ましい。

- IP受信ペイロード・メモリは、ポートip_payloadsmem_{addr, ce, we, wdata, rdata}と接続する。
- IP受信ペイロード・メモリは、以下の仕様とすること。
 - ポート数: 1RW
 - データ幅: 8ビット
 - アドレス幅: $\lceil \log_2(\text{PAYLOADSMEM_DEPTH}) \rceil$ ビット
 - ワード数: PAYLOADSMEM_DEPTH
 - クロックに同期して読み書きを行う
 - 以下のポートを持つ
 - ce: チップイネーブル(1でアサート)
 - we: ライト・イネーブル(1でアサート)
 - addr: アドレス
 - wdata: 書き込みデータ
 - rdata: 読み出しデータ(出力)
 - ceがアサート(1と)されているとき、アドレスaddrのデータをrdataに出力する。
 - ceがアサート(1と)されていて、かつweがアサート(1と)されているとき、アドレスaddrにwdataの値を書き込む。

3.9. Ethernet フレーム送信メモリの用意と接続

Ethernet フレーム送信メモリは、ROS2rapperが送信する Ethernet フレームを保持するRAMである。

Ethernet フレーム送信メモリは、ユーザ・ロジックとして、メモリまたはD-FFで実現して用意し接続する。

Ethernet フレーム送信メモリは、ポート tx_raw_eth_data_{addr, ce, rdata}と接続する。
なお、Ethernet フレーム送信メモリへの書き込みには、後述の手順が必要である。

ROS2rapper に接続する Ethernet フレーム送信メモリのインターフェースは、以下の仕様とすること。

- ポート数: 1R
- データ幅: 32ビット
- アドレス幅: $\lceil \log_2(\text{ROS2_MAX_RAW_ETH_TX_DATA_LEN}/4) \rceil$ ビット
- ワード数: $\text{ROS2_MAX_RAW_ETH_TX_DATA_LEN}/4$
- クロックに同期して読み出しを行う
- 以下のポートを持つ
 - ce: チップイネーブル(1でアサート)
 - addr: アドレス
 - rdata: 読み出しデータ(出力)
- ceがアサート(1)されているとき、アドレスaddrのデータをrdataに出力する。

3.10. Ethernet フレーム受信メモリの用意と接続

Ethernet フレーム受信メモリは、受信した Ethernet フレームを保持するRAMである。
ROS2rapper IPは、外部から受信した Ethernet フレームを Ethernet フレーム受信メモリに格納する。

Ethernet フレーム受信メモリは、ユーザ・ロジックとして、メモリまたはD-FFで実現して用意し接続する。

Ethernet フレーム受信メモリは、ポート`rx_raw_eth_data_{addr, ce, we, wdata}`と接続する。
なお、Ethernet フレーム受信メモリの読み出しには、後述の手順が必要である。

ROS2rapper に接続する Ethernet フレーム受信メモリのインターフェースは、以下の仕様とすること。

- ポート数: 1W
- データ幅: 32ビット
- アドレス幅: $\$clog2(ROS2_MAX_RAW_ETH_RX_DATA_LEN/4)$ ビット
- ワード数: $ROS2_MAX_RAW_ETH_RX_DATA_LEN/4$
- クロックに同期して読み書きを行う
- 以下のポートを持つ
 - `ce`:チップイネーブル(1でアサート)
 - `we`:ライト・イネーブル(1でアサート)、4 bit 幅 (バイト・イネーブル)
 - `addr`: アドレス
 - `wdata`: 書き込みデータ
- `ce`がアサート(1)されているとき、
 - `we[0]` がアサート (1) されていれば、アドレス `addr` の第 0 ~ 7 bit に `wdata[7:0]` を書き込む。
 - `we[1]` がアサート (1) されていれば、アドレス `addr` の 8 ~ 15 bit に `wdata[15:8]` を書き込む。
 - `we[2]` がアサート (1) されていれば、アドレス `addr` の 16 ~ 23 bit に `wdata[23:16]` を書き込む。
 - `we[3]` がアサート (1) されていれば、アドレス `addr` の 24 ~ 31 bit に `wdata[31:24]` を書き込む。

3.11. ノード情報格納メモリの用意と接続

ノード情報格納メモリ (Participant Table) は、RTPS で発見した通信相手のノード情報を保持するRAMである。

ノード情報格納メモリは、ユーザ・ロジックとして、メモリまたはD-FFで実現して用意し接続する。ノード情報格納メモリは、ポート `sedp_reader_tbl_mem_{addr, ce, we, rdata, wdata}` と接続する。

ノード情報格納メモリは、以下の仕様とすること。

- ポート数: 1RW
- データ幅: 64ビット
- アドレス幅: $\lceil \log_2(\text{ROS2_SEDP_READER_MAX} \times 11) \rceil$ ビット
- ワード数: $\text{ROS2_SEDP_READER_MAX} \times 11$
- クロックに同期して読み書きを行う
- 以下のポートを持つ
 - `ce`: チップイネーブル(1でアサート)
 - `we`: ライト・イネーブル(1でアサート)
 - `addr`: アドレス
 - `wdata`: 書き込みデータ
 - `rdata`: 読み出しデータ(出力)
- `ce`がアサート(1)されているとき、アドレス`addr`のデータを`rdata`に出力する。
- `ce`がアサート(1)されているとき、`we` がアサート (1) されていれば、アドレス `addr` に `wdata` を書き込む。

3.12. トピック情報格納メモリの用意と接続

トピック情報格納メモリ (Endpoint Table) は、RTPS で発見した通信相手のトピック毎の情報を保持するRAMである。

トピック情報格納メモリは、ユーザ・ロジックとして、メモリまたはD-FFで実現して用意し接続する。

トピック情報格納メモリは、ポート `app_reader_tbl_mem_{addr, ce, we, rdata, wdata}` と接続する。

トピック情報格納メモリは、以下の仕様とすること。

- ポート数: 1RW
- データ幅: 64ビット
- アドレス幅: $\lceil \log_2(\text{ROS2_APP_READER_MAX}) \rceil$ ビット
- ワード数: `ROS2_APP_READER_MAX`
- クロックに同期して読み書きを行う
- 以下のポートを持つ
 - `ce`: チップイネーブル(1でアサート)
 - `we`: ライト・イネーブル(1でアサート)
 - `addr`: アドレス
 - `wdata`: 書き込みデータ
 - `rdata`: 読み出しデータ(出力)
- `ce`がアサート(1)されているとき、アドレス`addr`のデータを`rdata`に出力する。
- `ce`がアサート(1)されているとき、`we` がアサート (1) されていれば、アドレス `addr` に `wdata` を書き込む。

4. ROS2通信

4.1. ROS2rapper各機能の有効化

先述のとおり、本ROS2rapper IPは、下記の各機能を独立して個々に有効化/無効化できる。

- ROS2パブリッシュ機能
- ROS2サブスクライブ機能
- Ethernet

ROS2rapperの機能を有効化する前に、Ethernet MACなど物理層の有効化や初期化が必要である。

つまり、本実装サンプルでは先述のポートの設定を行った上で、Ether MAC(物理通信手段)を有効化した後、ROS2rapperのROS2のパブリッシュ、あるいはサブスクライブ機能の有効化を行う。

機能の有効化は次のとおり。

- トピックのパブリッシュを有効にする(すなわちパブリッシャとして使用する)には `ros2pub_en` の有効化したいトピックに対応する bit を、アサート(1と)する。
- サブスクライブを有効にする(すなわちサブスクライバとして使用する)には `ros2sub_en` の有効化したいトピックに対応する bit を、それぞれアサート(1と)する。
- パブリッシュとサブスクライブは独立に個々に、または同時に使用できる
- Ethernet MACを有効化するには、ポートの `ether_en` をアサート(1と)する。
 - `ether_en` を一度アサートしたら、デアサート(0と)してはならない。

なお、サンプル実装では、`ros2_ether`と接続するポートはレジスタとしてユーザ・ロジックの中に実装して、接続している。

4.2. パブリッシャの送信メッセージ(アプリケーション・データ・メモリ)の変更

パブリッシャ有効時には、ユーザ・ロジック内に用意された、アプリケーション・データ・メモリ (送信) からのデータを、ROS2送信メッセージとして自動的に周期的に送出する。
送信内容(アプリケーション・データ・メモリの値)を変更するためには、後述の手続きが必要である。

ユーザ・ロジックは、アプリケーション・データ・メモリ (送信) を変更する前に、ROS2rapperより「メッセージ変更権」を取得する必要がある。これは、変更途中のメッセージがROS2rapperにより読み出され、送信されることを防ぐためである。

下記のポートが、メッセージ変更権の制御のために用意されている。ポートはいずれも 4 bit 幅であり、各 bit が 4 つのトピックそれぞれに対応している。

- `ros2_pub_app_data_req`(ROS2rapperへの入力): メッセージ変更権を要求する
- `ros2_pub_app_data_grant`(ROS2rapperからの出力):メッセージ変更権の有無
- `ros2_pub_app_data_ack`(ROS2rapperからの出力): メッセージ変更権が与えられた
- `ros2_pub_app_data_nack`(ROS2rapper からの出力): メッセージ変更が拒否された
- `ros2_pub_app_data_rel`(ROS2rapperの入力): メッセージ変更権を解放

送信メッセージ(`ros2_pub_app_data`入力)変更時のポート操作手順を以下に記述する。以下の操作は、各ポートの、変更したいトピックに対応する bit に対して行う。

1. 上記の 5 つのポートの (変更したいトピックに) 対応する bit を確認する。
 - a. いずれかが 1 の場合は、操作中のため、まず下記の手順にしたがって操作を完了させる。
 - b. 全て 0 の場合は、下記の手順に従ってメッセージの変更を行う。
2. `ros2_pub_app_data_req` の対応する bit をアサート(1と)して、メッセージ変更権を要求する。
3. `ros2_pub_app_data_ack` または `ros2_pub_app_data_nack` の対応する bit が 1 となるのを待つ。
その後、`ros2_pub_app_data_req` の対応する bit をデアサート (0 と) し、
`ros2_pub_app_data_ack` または `ros2_pub_app_data_nack` の対応する bit が 0 となるのを待つ。
 - a. `ros2_pub_app_data_ack` の対応する bit が 1 となった場合は、メッセージ変更権が与えられたため、次の手順に進む。
メッセージ変更は `ros2_pub_app_data_ack` の対応する bit が 0 となるのを待たずに行ってよい。
 - b. `ros2_pub_app_data_nack` の対応する bit が 1 となった場合は、ROS2rapper がメッセージを送信中であり、メッセージ更新が拒否されている。このときは、`ros2_pub_app_data_nack` の対応する bit が 0 になるまで待って、始めからやり直す。
4. メッセージの変更を行う。
5. `ros2_pub_app_data_rel` の対応する bit をアサート (1 と) して、
`ros2_pub_app_data_grant` の対応する bit が 0 になるのを待つ。
`ros2_pub_app_data_grant` の対応する bit が 0 になったら、`ros2_pub_app_data_rel` の対応する bit をデアサート (0 と) する。

送信アプリケーション・データ・メモリに書き込むアプリケーション・データのフォーマットは、Extensible and Dynamic Topic Types for DDS 仕様(参考文献[2])の、Classic CDR representation with Little Endian encodingに従うこと。エンコーディングの詳細は、参考文献[2] 7.4.3.5節のエンコーディング・ルールを参照すること。

1バイト中のビットの並びはLSB 0である。バイト・オーダーはリトル・エンディアンでなければならない。

例) 32ビット整数x2(0x12345678, 0xaabbaabb)を含むアプリケーション・データ

Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
78	56	34	12	bb	aa	bb	aa

32ビット整数 "0x12345678" 32ビット整数 "0xaabbaabb"

例) 文字列"hello, world!"を含むアプリケーション・データ

Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7	Byte 8	Byte 9	Byte 10	Byte 11	Byte 12	Byte 13	Byte 14	Byte 15	Byte 16
0d	00	00	00	68	65	6c	6c	6f	2c	20	77	6f	72	6c	64	21

文字列長 13 文字列 "hello, world!"

4.3. サブスクライバの受信メッセージ(アプリケーション・データ・メモリ)の読み出し

サブスクライバ機能は、サブスクライブしたデータを受信し、メモリ・インターフェースを通して、ユーザ・ロジック内に用意されたアプリケーション・データ・メモリに受信データを書き込む。

受信したメッセージを読み出す際に「メッセージ読み出し権」の取得が必要となる。これは、ユーザ・ロジックのメッセージ読み出し中に、ROS2rapper IPが、次の受信メッセージでメモリを上書きしないための手順である。

下記のポートが、メッセージ変更権の制御のために用意されている。ポートはいずれも 4 bit 幅であり、各 bit が 4 つのトピックそれぞれに対応している。

- `ros2_sub_app_data_req`(ROS2rapperの入力): メッセージ読み出し権を要求する
- `ros2_sub_app_data_grant`(ROS2rapperからの出力): メッセージ読み出し権の有無
- `ros2_sub_app_data_ack`(ROS2rapperからの出力): メッセージ読み出し権が与えられた
- `ros2_sub_app_data_nack`(ROS2rapperからの出力): メッセージ読み出しが拒否された
- `ros2_sub_app_data_rel`(ROS2rapperの入力): メッセージ読み出し権を解放

受信メッセージ読み出し時のポート操作手順を以下に述べる。以下の操作は、各ポートの、読み出したいトピックに対応する bit に対して行う。

1. 上記の 5 つのポートの (読み出したいトピックに) 対応する bit を確認する。
 - a. いずれかが 1 の場合は、操作中のため、まず下記の手順にしたがって操作を完了させる。
 - b. 全て 0 の場合は、下記の手順に従ってメッセージの変更を行う。
2. `ros2_sub_app_data_req` の対応する bit をアサート(1と)して、メッセージ読み出し権を要求する。
3. `ros2_sub_app_data_ack` または `ros2_sub_app_data_nack` の対応する bit が 1 となるのを待つ。
その後、`ros2_sub_app_data_req` の対応する bit をデアサート (0 と) し、
`ros2_sub_app_data_ack` または `ros2_sub_app_data_nack` の対応する bit が 0 となるのを待つ。
 - a. `ros2_sub_app_data_ack` の対応する bit が 1 となった場合は、メッセージ読み出し権が与えられたため、次の手順に進む。
メッセージ読み出しは `ros2_sub_app_data_ack` の対応する bit が 0 となるのを待たずに行ってもよい。
 - b. `ros2_sub_app_data_nack` の対応する bit が 1 となった場合は、ROS2rapper がメッセージを受信中であり、メッセージ読み出しが拒否されている。このときは、`ros2_sub_app_data_nack` の対応する bit が 0 になるまで待って、始めからやり直す。
4. メッセージの変更を行う。
5. `ros2_sub_app_data_rel` の対応する bit をアサート (1 と) して、
`ros2_sub_app_data_grant` の対応する bit が 0 になるのを待つ。
`ros2_sub_app_data_grant` の対応する bit が 0 になったら、`ros2_sub_app_data_rel` の対応する bit をデアサート (0 と) する。

アプリケーション・データの受信時に、FIFO ポート `ros2_sub_app_data_recvinfo_{din, full_n, write}` から受信データのメタ情報を出し、データの受信を通知する。このとき、出力される情報は次の Fig. 4.1 の形式である：

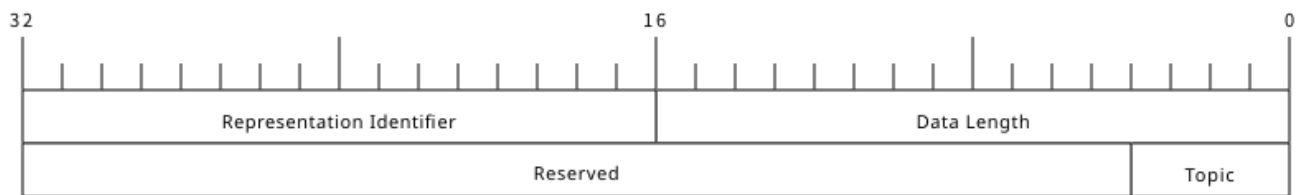


Fig 4.1. `ros2_sub_app_data_recvinfo` の形式

Data Length:

受信データ長 [bytes]

Representation Identifier:

受信データのフォーマットやバイト・オーダ等の判別に使用する。Representation Identifierの値については、Table 4.1 を参照すること。また、アプリケーション・データのフォーマットについては、参考文献[2] 7.4節を参照すること。

Topic:

どのトピックのデータを受信したかを表すビットマップ。

Table 4.1. Representation Identifier の一覧

Object Management Group, *The Real-time Publish-Subscribe Protocol DDS Interoperability Wire Protocol (DDS-I-RTPS™) Specification Version 2.3* (参考文献 [3]) より Table 10.3 を引用した。

引用の際、表に含まれる文献番号を本文書の参考文献リストの番号に合わせて変更した。

RepresentationIdentifier (see DDS-XTYPES Table 60)	Value	Representation Format
CDR_BE	{0x00, 0x00}	Classic CDR representation with Big Endian encoding. See DDS-XTypes [2] clause 7.4.1.1.
CDR_LE	{0x00, 0x01}	Classic CDR representation with Little Endian encoding. See DDS-XTypes [2] clause 7.4.1.1.
PL_CDR_BE	{0x00, 0x02}	ParameterList (9.4.2.11) with Big Endian encoding. See also DDS-XTypes [2] clause 7.4.1.2.
PL_CDR_LE	{0x00, 0x03}	ParameterList (9.4.2.11) with Little Endian encoding. See also DDS-XTypes [2] clause 7.4.1.2.
CDR2_BE	{0x00, 0x10}	Plain CDR representation (version2) with Big Endian encoding. Similar to Classic CDR except it uses a

		maximum alignment of 4 bytes. See DDS-XTypes [2] clause 7.4.2 and 7.4.3.
CDR2_LE	{0x00, 0x11}	Plain CDR representation (version2) with Little Endian encoding. Similar to Classic CDR except it uses a maximum alignment of 4 bytes. See DDS-XTypes [2] clause 7.4.2 and 7.4.3.
PL_CDR2_BE	{0x00, 0x12}	Extended CDR representation (version2) for MUTABLE types with Big Endian encoding. A generalization of ParameterList. See DDS-XTypes [2] clause 7.4.2 and 7.4.3.
PL_CDR2_LE	{0x00, 0x13}	Extended CDR representation (version2) for MUTABLE types with Little Endian encoding. A generalization of ParameterList. See DDS-XTypes [2] clause 7.4.2 and 7.4.3.
D_CDR_BE	{0x00, 0x14}	Extended CDR representation (version2) for APPENDABLE types with Big Endian encoding. Similar to plain CDR2_BE except for a delimiter. See DDS-XTypes [2] clause 7.4.2 and 7.4.3.
D_CDR_LE	{0x00, 0x15}	Extended CDR representation (version2) for APPENDABLE types with Little Endian encoding. Similar to plain CDR2_BE except for a delimiter. See DDS-XTypes [2] clause 7.4.2 and 7.4.3.
XML	{0x00, 0x04}	See DDS-XTypes [2] clause 7.4.4.

5. Ethernet フレームの通信

5.1. Ethernet フレームの送信

ROS2rapperでは、Ethernet フレームを直接送信することができる。

下記のポートが、Ethernet フレームの送信のために用意されている。

- tx_raw_eth_frame_ready (ROS2rapper への入力): Ethernet フレームを送信する。
- tx_raw_eth_completed (ROS2rapper の出力): Ethernet フレームの送信が完了した。

以下、手順を述べる。

1. tx_raw_eth_frame_ready と tx_raw_eth_completed を確認する。
 - a. いずれかが 1 のときは、操作中なので、下記の手順に従って操作を完了させる。
 - b. とともに 0 のときは、下記の手順にしたがって Ethernet フレームの送信を行う。
2. Ethernet フレーム送信メモリに送信する Ethernet フレームを書き込む。
3. tx_raw_eth_frame_ready をアサート (1 と) する。(送信が開始される)
4. 送信が完了すると tx_raw_eth_completed が 1 になる。その後、tx_raw_eth_frame_ready をデアサート (0 と) し、tx_raw_eth_completed が 0 になるのを待つ。

Ethernet フレーム送信メモリ中のデータの形式は、次の Fig. 5.1 の通りである

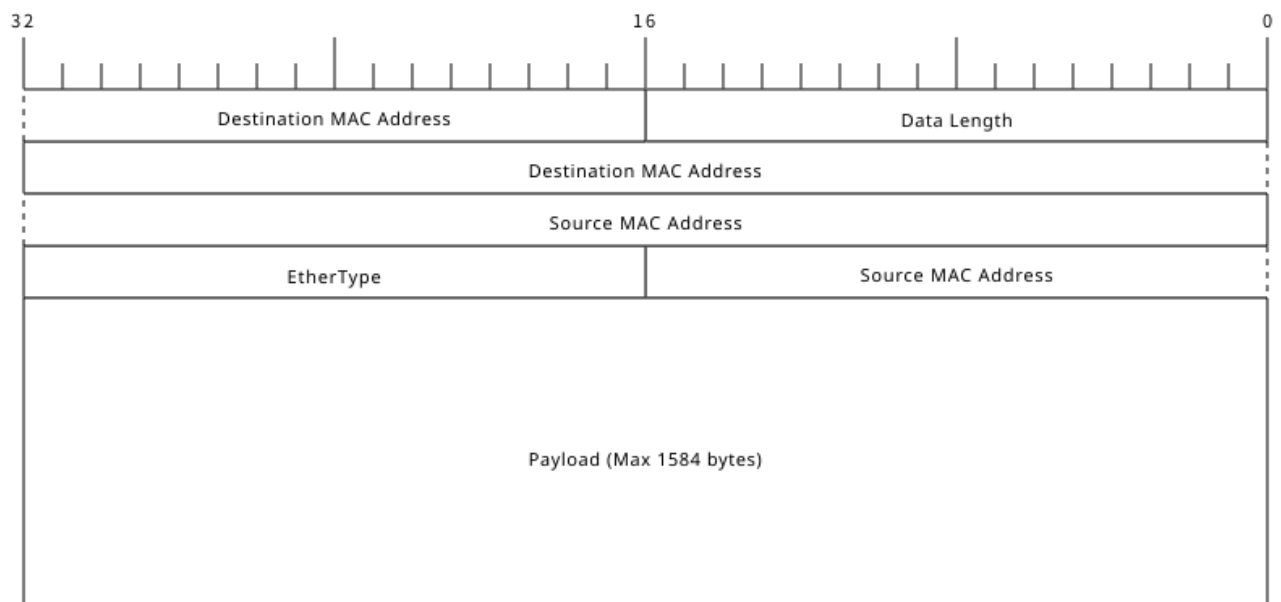


Fig. 5.1. Ethernet フレームメモリ中のデータの形式

Data Lenth:

送信フレーム長 [bytes] (little endian、ヘッダの長さを含む)。
`ROS2\_MAX\_RAW\_ETH\_TX\_DATA\_LEN-2` 以下でなければならない。

Destination MAC Address:

宛先の MAC アドレス

Source MAC Address:

送信元の MAC アドレス

EtherType:

Ethernet フレームの種類

つまり、先頭 2 bytes のフレーム長の後ろに、送信する Ethernet フレームが続く、という形式である。

EtherType が IPv4 を表す 0x0800 (big endian) の場合は、IP パケット送信モジュールの機能を利用して、Ethernet フレームの送信が行われる。このため、Ethernet フレーム送信メモリ中のデータは次の Fig. 5.2 の形式でなければならない:

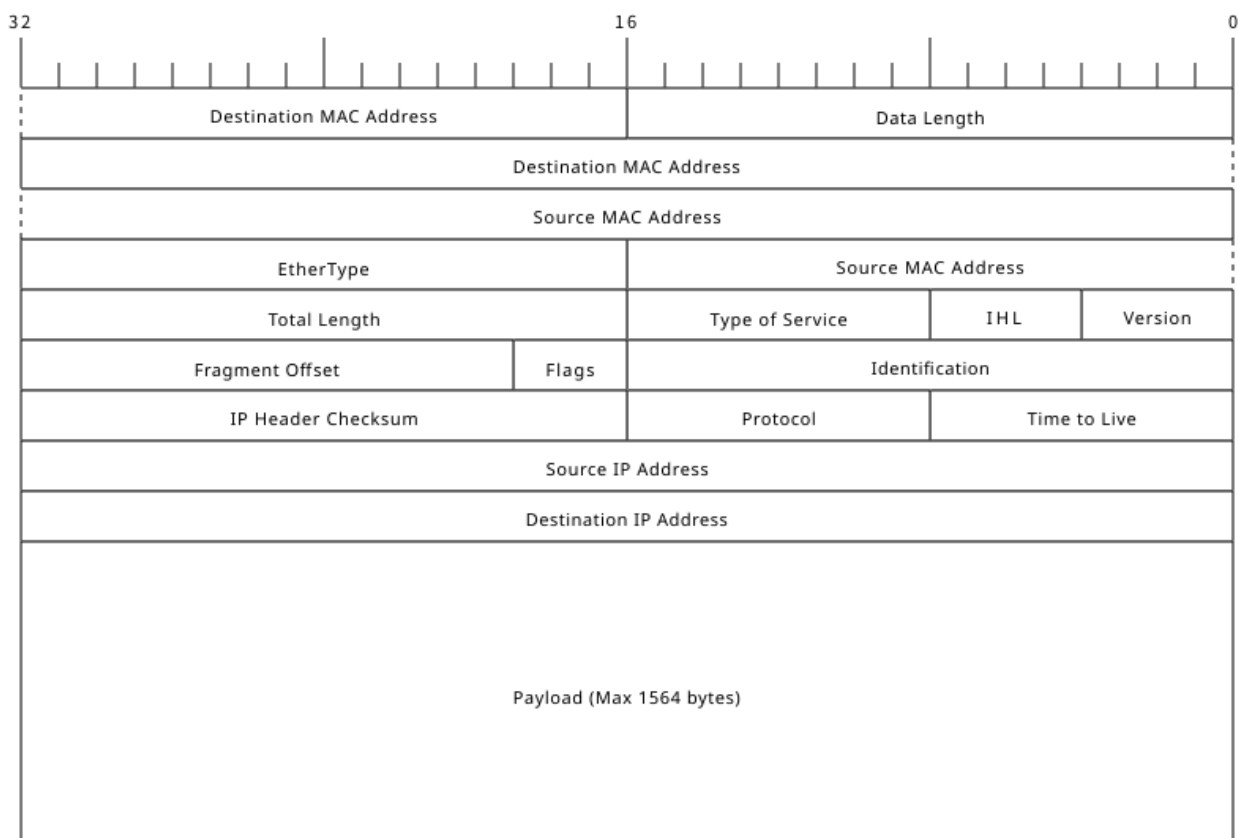


Fig. 5.2. IPv4 パケット送信時の Ethernet フレーム送信メモリのデータの形式

IPv4 パケットの送信時、以下の項目は無視・上書きされる。特に、IP ヘッダにオプションを含められず、IP フラグメントの送信を行えない、という制約が存在する。

Destination MAC Address:

Source IP Address に基づいて、ARP モジュールで解決される。

Source MAC Address:

ROS2rapper のポート mac_addr に設定した値が使用される。

Version:

“4” が使用される。

IHL:

“5” が使用される。IP ヘッダにオプションを含めることはできない。

Identification:

0x0000 が使用される。

Flags:

0b010 (Don't Fragment) が使用される。IP フラグメントを送信することはできない。

IP Header Checksum:

IP パケット送信モジュールで計算した値が使用される。

5.2. Ethernet フレームの受信

ROS2rapper では、受信した ARP・RTPS パケット以外の Ethernet フレームを外部のメモリ (Ethernet フレーム受信メモリ) に出力する。メモリに出力する正確な条件は以下の通りである：

1. 前に出力したフレームが未確認で残っているときは、受信した Ethernet フレームは出力せず破棄する。
2. ARP パケットは出力しない。
3. IPv4 パケットであって、宛先の IP アドレスが以下のいずれかを満たすものは、出力しない。
 - a. ユニキャストであって、ROS2rapper に設定した IP アドレスと一致しない。
 - b. ブロードキャストであって、その宛先が ROS2rapper に設定した IP アドレスを含まない。
4. UDP/IPv4 パケットであって、UDP ペイロードの先頭 4 文字が “RTPS” に一致するものは、出力しない。ただし、IP ヘッダにオプションが含まれるか、IP パケットのフラグメントである場合は出力される。

下記のポートが、Ethernet フレーム受信のために用意されている。

- rx_raw_eth_data_frame_ready (ROS2rapper からの出力): Ethernet フレーム受信メモリにデータを置いた後、アサート (1 と) する。
 - このとき、Ethernet フレーム受信メモリからデータを読み出すことが可能である。
 - このポートが 1 になっている間は、新たに受信した Ethernet フレームは、Ethernet フレーム受信メモリに書き込まれず、破棄される。
- rx_raw_eth_data_ack (ROS2rapper への入力): 新たな Ethernet フレームの書き込みを可能にする。

以下、手順を述べる。

1. Ethernet フレームが受信され、Ethernet フレーム受信メモリに置かれると、rx_raw_eth_data_frame_ready がアサート(1と)される
2. Ethernet フレーム受信メモリからデータを読み出す
3. メモリからの読出しが完了したら、新しい Ethernet フレームの書き込みを可能にするため、ユーザ・ロジックは、rx_raw_eth_data_ack をアサート (1 と) し、rx_raw_eth_data_frame_ready 0 になるまで待つ。
4. rx_raw_eth_data_frame_ready が 0 になったら、rx_raw_eth_data_ack を 0 とする。

Ethernet フレーム受信メモリに置かれるデータの形式は、Ethernet フレーム送信メモリと同じ Fig. 5.1 の形式である。

6. 参考文献

- [1] Topic and Service name mapping to DDS
(https://design.ros2.org/articles/topic_and_service_names.html#mapping-of-ros-2-topic-and-service-names-to-dds-concepts)
- [2] Extensible and Dynamic Topic Types for DDS
(<https://www.omg.org/spec/DDS-XTypes/1.3/PDF>)
- [3] The Real-time Publish-Subscribe Protocol DDS Interoperability Wire Protocol (DDSI-RTPS™) Specification Version 2.3
(<https://www.omg.org/spec/DDSI-RTPS/2.3/PDF>)
- [4] DDS RTPS Vendor and Protocol IDs
(<https://www.dds-foundation.org/dds-rtps-vendor-and-product-ids/>)
2026年3月13日閲覧
- [5] Alex Forencich, verilog-ethernet, 2019.
(<https://github.com/alexforencich/verilog-ethernet>)
2026年3月16日閲覧

7. 付録

7.1. ros2_etherモジュールのインターフェース仕様

ros2_etherモジュールのポート一覧をTable A-1に示す。

- 各信号は、LSB 0である。
- 接尾辞が_nの信号は負論理、それ以外は正論理である。
 - 正論理の信号は、アサートで信号を1に、デアサートで信号を0にする。
 - 負論理の信号は、アサートで信号を0に、デアサートで信号を1にする。

Table A-1 ros2_etherモジュールのポート仕様一覧

ポート名	方向	ビット幅	説明
クロック・リセット			
clk	input	1	クロック
rst_n	input	1	リセット(0でアサート)
機能有効化			
ether_en	input	1	Ethernet MACイネーブル 1: Ethernet有効 0: Ethernet無効 1に設定後、再び0に設定した場合の動作は保証されない。
ros2pub_en	input	`ROS2_PUB_TOPIC_MAX	トピック・パブリッシュ・イネーブル 1: トピック・パブリッシュ有効 0: トピック・パブリッシュ無効 それぞれのビットが1つのトピックに対応している。 1に設定した後、再び0に設定した場合の動作は保証されない。
ros2sub_en	input	`ROS2_SUB_TOPIC_MAX	トピック・サブスクライブ・イネーブル 1: トピック・サブスクライブ有効 0: トピック・サブスクライブ無効 それぞれのビットが1つのトピックに対応している。 1に設定した後、再び0に設定した場合の動作は保証されない。
Ethernet MII			
phy_rx_clk	input	1	Ethernet MII 受信クロック
phy_rxd	input	4	Ethernet MII 受信データ
phy_rx_dv	input	1	Ethernet MII受信データ有効
phy_rx_er	input	1	Ethernet MII 受信エラー
phy_tx_clk	input	1	Ethernet MII 送信クロック

phy_txd	output	4	Ethernet MII 送信データ
phy_tx_en	output	1	Ethernet MII 送信イネーブル
phy_rst_n	output	1	Ethernet MII リセット
Ethernetアドレス/IPアドレス			
mac_addr	input	48	MACアドレス
ip_addr	input	32	IPアドレス a.b.c.dを設定する場合、ip_addr[31:0] = {d,c,b,a}とする。
gateway_ip_addr	input	32	ゲートウェイIPアドレス
subnet_mask	input	32	サブネットマスク
ros2_fragment_expiration	input	32	IPフラグメントがすべて揃わない際、IP フラグメントを保持する最小の時間。 ROS2rapper内部サイクル数で指定す る。
ARP			
arp_req_retry_count	input	6	ARPリトライ回数
arp_req_retry_interval	input	36	ARPリトライ間隔(サイクル数)
arp_req_timeout	input	36	ARPタイムアウト時間(サイクル数)
ROS2			
ros2_vendor_id	input	16	RTPS のベンダー ID。参考文献 [4] 参 照のこと。
ros2_node_name	input	ROS2_MAX_NODE _NAME_LEN*8	ノード名 '\0'終端とし、終端以降はゼロで埋める こと。
ros2_node_name_len	input	8	ノード名の長さ '\0'を含めたバイト数で指定すること。
ros2_node_udp_port	input	16	ノードのUDPポート番号
ros2_port_num_seed	input	16	UDPポート番号のシード値 ROS2で使用するポート番号導出のた めのマジックナンバーであり次の計算 式で得られる。 (ROS2のDomain ID)*250+7400
ros2_guid_prefix	input	96	GUID(グローバル一意識別子) プレフィ クス
ros2_participant_lease_duration_ seconds	input	32	RTPS の participant lease duration の整数部 [sec] participant lease duration は SPDP の送信間隔より充分長くなければなら ない。
ros2_participant_lease_duration_f raction	input	32	RTPS の participant lease duration の小数部 [2^-32 sec] participant lease duration は SPDP の送信間隔より充分長くなければなら

			ない。
パブリッシュするトピック			
ros2_pub_topic_name_0	input	ROS2_MAX_TOPIC_NAME_LEN*8	トピック名 '\0'終端とし、終端以降はゼロで埋めること。
ros2_pub_topic_name_1	input	ROS2_MAX_TOPIC_NAME_LEN*8	同上
ros2_pub_topic_name_2	input	ROS2_MAX_TOPIC_NAME_LEN*8	同上
ros2_pub_topic_name_3	input	ROS2_MAX_TOPIC_NAME_LEN*8	同上
ros2_pub_topic_name_len_0	input	8	トピック名 ros2_pub_topic_name_len_0 の長さ '\0'を含めたバイト数で指定すること。
ros2_pub_topic_name_len_1	input	8	トピック名 ros2_pub_topic_name_len_1 の長さ '\0'を含めたバイト数で指定すること。
ros2_pub_topic_name_len_2	input	8	トピック名 ros2_pub_topic_name_len_2 の長さ '\0'を含めたバイト数で指定すること。
ros2_pub_topic_name_len_3	input	8	トピック名 ros2_pub_topic_name_len_3 の長さ '\0'を含めたバイト数で指定すること。
ros2_pub_topic_type_name_0	input	ROS2_MAX_TOPIC_TYPE_NAME_LEN*8	トピック型名 '\0'終端とし、終端以降はゼロで埋めること。
ros2_pub_topic_type_name_1	input	ROS2_MAX_TOPIC_TYPE_NAME_LEN*8	同上
ros2_pub_topic_type_name_2	input	ROS2_MAX_TOPIC_TYPE_NAME_LEN*8	同上
ros2_pub_topic_type_name_3	input	ROS2_MAX_TOPIC_TYPE_NAME_LEN*8	同上
ros2_pub_topic_type_name_len_0	input	8	トピック型名 ros2_pub_topic_type_name_0 の長さ '\0'を含めたバイト数で指定すること。
ros2_pub_topic_type_name_len_1	input	8	トピック型名 ros2_pub_topic_type_name_1 の長さ '\0'を含めたバイト数で指定すること。
ros2_pub_topic_type_name_len_2	input	8	トピック型名 ros2_pub_topic_type_name_2 の長さ

			さ '\0'を含めたバイト数で指定すること。
ros2_pub_topic_type_name_len_3	input	8	トピック型名 ros2_pub_topic_type_name_3 の長さ '\0'を含めたバイト数で指定すること。
ros2_pub_app_data_0_addr	output	$\$clog2(\text{'ROS2_MAX_APP_DATA_LEN/4'})$	ユーザ・ロジックに確保した、送信アプリケーション・データ・メモリのアドレス
ros2_pub_app_data_0_ce	output	1	送信アプリケーション・データ・メモリのチップ・イネーブル
ros2_pub_app_data_0_rdata	input	32	送信アプリケーション・データ・メモリの読み出しデータ
ros2_pub_app_data_1_addr	output	$\$clog2(\text{'ROS2_MAX_APP_DATA_LEN/4'})$	ユーザ・ロジックに確保した、送信アプリケーション・データ・メモリのアドレス
ros2_pub_app_data_1_ce	output	1	送信アプリケーション・データ・メモリのチップ・イネーブル
ros2_pub_app_data_1_rdata	input	32	送信アプリケーション・データ・メモリの読み出しデータ
ros2_pub_app_data_2_addr	output	$\$clog2(\text{'ROS2_MAX_APP_DATA_LEN/4'})$	ユーザ・ロジックに確保した、送信アプリケーション・データ・メモリのアドレス
ros2_pub_app_data_2_ce	output	1	送信アプリケーション・データ・メモリのチップ・イネーブル
ros2_pub_app_data_2_rdata	input	32	送信アプリケーション・データ・メモリの読み出しデータ
ros2_pub_app_data_3_addr	output	$\$clog2(\text{'ROS2_MAX_APP_DATA_LEN/4'})$	ユーザ・ロジックに確保した、送信アプリケーション・データ・メモリのアドレス
ros2_pub_app_data_3_ce	output	1	送信アプリケーション・データ・メモリのチップ・イネーブル
ros2_pub_app_data_3_rdata	input	32	送信アプリケーション・データ・メモリの読み出しデータ
ros2_pub_app_data_len_0	input	$\text{'ROS2_APP_DATA_LEN_WIDTH'}$	ros2_pub_app_data_0 のメッセージの長さ バイト数で指定すること。
ros2_pub_app_data_len_1	input	$\text{'ROS2_APP_DATA_LEN_WIDTH'}$	ros2_pub_app_data_1 のメッセージの長さ バイト数で指定すること。
ros2_pub_app_data_len_2	input	$\text{'ROS2_APP_DATA_LEN_WIDTH'}$	ros2_pub_app_data_2 のメッセージの長さ バイト数で指定すること。
ros2_pub_app_data_len_3	input	$\text{'ROS2_APP_DATA_LEN_WIDTH'}$	ros2_pub_app_data_3 のメッセージの長さ

			バイト数で指定すること。
ros2_pub_app_data_req	input	`ROS2_PUB_TOPIC_MAX	メッセージ変更権リクエスト 詳細は「4.2. パブリッシャの送信メッセージ(アプリケーション・データ・メモリ)の変更」を参照のこと
ros2_pub_app_data_rel	input	`ROS2_PUB_TOPIC_MAX	メッセージ変更権解放 詳細は「4.2. パブリッシャの送信メッセージ(アプリケーション・データ・メモリ)の変更」を参照のこと
ros2_pub_app_data_grant	output	`ROS2_PUB_TOPIC_MAX	メッセージ変更権 詳細は「4.2. パブリッシャの送信メッセージ(アプリケーション・データ・メモリ)の変更」を参照のこと
ros2_pub_app_data_ack	output	`ROS2_PUB_TOPIC_MAX	詳細は「4.2. パブリッシャの送信メッセージ(アプリケーション・データ・メモリ)の変更」を参照のこと
ros2_pub_app_data_nack	output	`ROS2_PUB_TOPIC_MAX	同上
サブスクライブするトピック			
ros2_sub_topic_name_0	input	ROS2_MAX_TOPIC_NAME_LEN*8	トピック名 `0'終端とし、終端以降はゼロで埋めること。
ros2_sub_topic_name_1	input	ROS2_MAX_TOPIC_NAME_LEN*8	同上
ros2_sub_topic_name_2	input	ROS2_MAX_TOPIC_NAME_LEN*8	同上
ros2_sub_topic_name_3	input	ROS2_MAX_TOPIC_NAME_LEN*8	同上
ros2_sub_topic_name_len_0	input	8	トピック名 ros2_sub_topic_name_0 の長さ `0'を含めたバイト数で指定。
ros2_sub_topic_name_len_1	input	8	トピック名 ros2_sub_topic_name_1 の長さ `0'を含めたバイト数で指定。
ros2_sub_topic_name_len_2	input	8	トピック名 ros2_sub_topic_name_2 の長さ `0'を含めたバイト数で指定。
ros2_sub_topic_name_len_3	input	8	トピック名 ros2_sub_topic_name_3 の長さ `0'を含めたバイト数で指定。
ros2_sub_topic_type_name_0	input	ROS2_MAX_TOPIC_TYPE_NAME_LEN*8	トピック型名 `0'終端とし、終端以降はゼロで埋めること。
ros2_sub_topic_type_name_1	input	ROS2_MAX_TOPIC_TYPE_NAME_LEN	同上

		N*8	
ros2_sub_topic_type_name_2	input	ROS2_MAX_TOPIC _TYPE_NAME_LE N*8	同上
ros2_sub_topic_type_name_3	input	ROS2_MAX_TOPIC _TYPE_NAME_LE N*8	同上
ros2_sub_topic_type_name_len_0	input	8	トピック型名 ros2_sub_topic_type_name_0 の長さ '\0'含めたバイト数で指定。
ros2_sub_topic_type_name_len_1	input	8	トピック型名 ros2_sub_topic_type_name_1 の長さ '\0'含めたバイト数で指定。
ros2_sub_topic_type_name_len_2	input	8	トピック型名 ros2_sub_topic_type_name_2 の長さ '\0'含めたバイト数で指定。
ros2_sub_topic_type_name_len_3	input	8	トピック型名 ros2_sub_topic_type_name_3 の長さ '\0'含めたバイト数で指定。
ros2_sub_app_data_req	input	`ROS2_SUB_TOPI CS_MAX	メッセージ読み出し権リクエスト 詳細は「4.3. サブスクリバの受信メッ セージ(アプリケーション・データ・メモリ) の読み出し」を参照のこと。
ros2_sub_app_data_rel	input	`ROS2_PUB_TOPI CS_MAX	メッセージ読み出し権解放 詳細は「4.3. サブスクリバの受信メッ セージ(アプリケーション・データ・メモリ) の読み出し」を参照のこと。
ros2_sub_app_data_grant	output	`ROS2_PUB_TOPI CS_MAX	メッセージ読み出し権 詳細は「4.3. サブスクリバの受信メッ セージ(アプリケーション・データ・メモリ) の読み出し」を参照のこと。
ros2_sub_app_data_ack	output	`ROS2_PUB_TOPI CS_MAX	詳細は「4.3. サブスクリバの受信メッ セージ(アプリケーション・データ・メモリ) の読み出し」を参照のこと。
ros2_sub_app_data_nack	output	`ROS2_PUB_TOPI CS_MAX	詳細は「4.3. サブスクリバの受信メッ セージ(アプリケーション・データ・メモリ) の読み出し」を参照のこと。
ros2_sub_app_data_recvinfo_din	output	64	受信したアプリケーション・データのメタ データ。詳細は「4.3. サブスクリバの 受信メッセージ(アプリケーション・デー タ・メモリ)の読み出し」を参照のこと。
ros2_sub_app_data_recvinfo_full_n	input	1	受信したアプリケーション・データのメタ データを受け取る FIFO が満杯かどうか

			か。 1 のとき、データを受け取れる。
ros2_sub_app_data_recvinfo_write	output		1 受信したアプリケーション・データのメタデータを受け取る FIFO のライト・イネーブル
ros2_sub_app_data_0_addr	output	\$clog2(`ROS2_MAX_APP_DATA_LEN)	アプリケーション・データ・メモリのアドレス ユーザ・ロジックに確保したメモリのアドレス
ros2_sub_app_data_0_ce	output		1 アプリケーション・データ・メモリのチップイネーブル
ros2_sub_app_data_0_we	output		1 アプリケーション・データ・メモリのライト・イネーブル
ros2_sub_app_data_0_wdata	output		8 アプリケーション・データ・メモリの書き込みデータ
ros2_sub_app_data_1_addr	output	\$clog2(`ROS2_MAX_APP_DATA_LEN)	アプリケーション・データ・メモリのアドレス ユーザ・ロジックに確保したメモリのアドレス
ros2_sub_app_data_1_ce	output		1 アプリケーション・データ・メモリのチップイネーブル
ros2_sub_app_data_1_we	output		1 アプリケーション・データ・メモリのライト・イネーブル
ros2_sub_app_data_1_wdata	output		8 アプリケーション・データ・メモリの書き込みデータ
ros2_sub_app_data_2_addr	output	\$clog2(`ROS2_MAX_APP_DATA_LEN)	アプリケーション・データ・メモリのアドレス ユーザ・ロジックに確保したメモリのアドレス
ros2_sub_app_data_2_ce	output		1 アプリケーション・データ・メモリのチップイネーブル
ros2_sub_app_data_2_we	output		1 アプリケーション・データ・メモリのライト・イネーブル
ros2_sub_app_data_2_wdata	output		8 アプリケーション・データ・メモリの書き込みデータ
ros2_sub_app_data_3_addr	output	\$clog2(`ROS2_MAX_APP_DATA_LEN)	アプリケーション・データ・メモリのアドレス ユーザ・ロジックに確保したメモリのアドレス
ros2_sub_app_data_3_ce	output		1 アプリケーション・データ・メモリのチップイネーブル
ros2_sub_app_data_3_we	output		1 アプリケーション・データ・メモリのライト・イネーブル
ros2_sub_app_data_3_wdata	output		8 アプリケーション・データ・メモリの書き込みデータ

Ethernet フレーム受信メモリ			
rx_raw_eth_data_addr	output	$\$clog2(\text{ROS2_MAX_RAW_ETH_RX_DATA_LEN}/4)$	Ethernet フレーム受信メモリのアドレス
rx_raw_eth_data_ce	output	1	Ethernet フレーム受信メモリのチップイネーブル
rx_raw_eth_data_we	output	4	Ethernet フレーム受信メモリのライト・イネーブル(バイト・イネーブル)
rx_raw_eth_data_wdata	output	32	Ethernet フレーム受信メモリの読み出しデータ
rx_raw_eth_data_ack	input	1	Ethernet フレーム受信メモリの読み出し権解放 詳細は「5.2. Ethernet フレームの受信」を参照のこと
rx_raw_eth_data_frame_ready	output	1	Ethernet フレーム受信メモリの読み出し権 詳細は「5.2. Ethernet フレームの受信」を参照のこと
Ethernet フレーム送信メモリ			
tx_raw_eth_data_addr	output	$\$clog2(\text{ROS2_MAX_RAW_ETH_TX_DATA_LEN}/4)$	Ethernet フレーム送信メモリのアドレス
tx_raw_eth_data_ce	output	1	Ethernet フレーム送信メモリのチップイネーブル
tx_raw_eth_data_rdata	input	32	Ethernet フレーム送信メモリの読み出しデータ
tx_raw_eth_frame_ready	input	1	Ethernet フレーム送信 詳細は「5.1. Ethernet フレームの送信」を参照のこと
tx_raw_eth_completed	output	1	Ethernet フレーム送信完了 詳細は「5.1. Ethernet フレームの送信」を参照のこと
IP受信ペイロード・メモリ			
ip_payloadsmem_addr	output	PAYLOADSMEM_AWIDTH	IP受信ペイロード・メモリのアドレス
ip_payloadsmem_ce	output	1	IP受信ペイロード・メモリのチップイネーブル
ip_payloadsmem_we	output	1	IP受信ペイロード・メモリのライト・イネーブル
ip_payloadsmem_wdata	output	8	IP受信ペイロード・メモリの書き込みデータ
ip_payloadsmem_rdata	input	8	IP受信ペイロード・メモリの読み出しデータ
ノード情報格納メモリ			

sedp_reader_tbl_mem_addr	output	\$clog2('ROS2_SED P_READER_MAX* 11)	ノード情報格納メモリのアドレス
sedp_reader_tbl_mem_ce	output	1	ノード情報格納メモリのチップイネーブル
sedp_reader_tbl_mem_we	output	1	ノード情報格納メモリのライト・イネーブル
sedp_reader_tbl_mem_wdata	output	64	ノード情報格納メモリの書き込みデータ
sedp_reader_tbl_mem_rdata	input	64	ノード情報格納メモリの読み出しデータ
トピック情報格納メモリ			
app_reader_tbl_mem_addr	output	\$clog2('ROS2_APP _READER_MAX*11)	トピック情報格納メモリのアドレス
app_reader_tbl_mem_ce	output	1	トピック情報格納メモリのチップイネーブル
app_reader_tbl_mem_we	output	1	トピック情報格納メモリのライト・イネーブル
app_reader_tbl_mem_wdata	output	64	トピック情報格納メモリの書き込みデータ
app_reader_tbl_mem_rdata	input	64	トピック情報格納メモリの読み出しデータ
パケット間隔に関連するポート			
ros2_timestamp_increment	input	32	クロック周期 [2 [^] -32 sec]
ros2_cnt_interval_elapsed	input	1	パケット送信開始
ros2_cnt_spdp_wr_elapsed	input	1	SPDP Writer のパケット送信開始
ros2_cnt_sedp_pub_wr_elapsed	input	1	SEDP Publications Writer のパケット送信開始
ros2_cnt_sedp_sub_wr_elapsed	input	1	SEDP Subscriptions Writer のパケット送信開始
ros2_cnt_sedp_pub_hb_elapsed	input	1	SEDP Publications Writer の HEARTBEAT パケット送信開始
ros2_cnt_sedp_sub_hb_elapsed	input	1	SEDP Subscriptions Writer の HEARTBEAT パケット送信開始
ros2_cnt_sedp_pub_an_elapsed	input	1	SEDP Publications Writer の ACKNACK パケット送信開始
ros2_cnt_sedp_sub_an_elapsed	input	1	SEDP Subscriptions Writer の ACKNACK パケット送信開始
ros2_cnt_app_elapsed	input	1	アプリケーション・データの送信開始
ros2_cnt_interval_set	output	1	パケット送信完了
ros2_cnt_spdp_wr_set	output	1	SPDP Writer のパケット送信完了
ros2_cnt_sedp_pub_wr_set	output	1	SEDP Publications Writer のパケット送信完了

ros2_cnt_sedp_sub_wr_set	output	1	SEDP Subscriptions Writer の パケット送信完了
ros2_cnt_sedp_pub_hb_set	output	1	SEDP Publications Writer の HEARTBEAT パケット送信完了
ros2_cnt_sedp_sub_hb_set	output	1	SEDP Subscriptions Writer の HEARTBEAT パケット送信完了
ros2_cnt_sedp_pub_an_set	output	1	SEDP Publications Writer の ACKNACK パケット送信完了
ros2_cnt_sedp_sub_an_set	output	1	SEDP Subscriptions Writer の ACKNACK パケット送信完了
ros2_cnt_app_set	output	1	アプリケーション・データの送信完了

Table A-2 パラメーター一覧

パラメータ名	説明	デフォルト値
SET_TX_PERIOD_BY_PARAMETER	パケット送信間隔を合成時のパラメータで設定するか。 0: 動作時にポートで制御 1: 合成時にパラメータで設定	0
PRESCALAR_DIV	パケット送信間隔カウンタの分周比	64
ROS2CLK_HZ	ROS2rapper の動作クロック周波数 [Hz]	100_000_000 (100MHz)
TX_INTERVAL_COUNT	パケットの最短送信間隔 (ROS2rapperのクロック数 / PRESCALER_DIV)	$(100_000_000 / \text{PRESCALER_DIV}) / 100$ (100 MHz 時 10msec)
TX_PERIOD_SPDP_WR_COUNT	SPDP Writerのパケット送信間隔 (ROS2rapperのクロック数 / PRESCALER_DIV)	$(100_000_000 / \text{PRESCALER_DIV}) * 3$ (クロック周波数 100 MHz で動作時 3sec)
TX_PERIOD_SEDP_PUB_WR_COUNT	SEDP Publication WriterのDATAパケット送信間隔 (ROS2rapperのクロック数 / PRESCALER_DIV)	$(100_000_000 / \text{PRESCALER_DIV}) * 3$ (クロック周波数 100 MHz で動作時 3sec)
TX_PERIOD_SEDP_SUB_WR_COUNT	SEDP Subscription WriterのDATAパケット送信間隔 (ROS2rapperのクロック数 / PRESCALER_DIV)	$(100_000_000 / \text{PRESCALER_DIV}) * 3$ (クロック周波数 100 MHz で動作時 3sec)
TX_PERIOD_SEDP_PUB_HB_COUNT	SEDP Publication Writerの HEARTBEATパケット送信間隔 (ROS2rapperのクロック数 / PRESCALER_DIV)	$(100_000_000 / \text{PRESCALER_DIV}) * 3$ (クロック周波数 100 MHz で動作時 3sec)

パラメータ名	説明	デフォルト値
TX_PERIOD_SEDP_SUB_HB_COUNT	SEDP Subscription WriterのHEARTBEATパケット送信間隔 (ROS2rapperのクロック数 / PRESCALER_DIV)	$(100_000_000 / \text{PRESCALER_DIV}) * 3$ (クロック周波数 100 MHz で動作時 3sec)
TX_PERIOD_SEDP_PUB_AN_COUNT	SEDP Publication WriterのACKNACKパケット送信間隔 (ROS2rapperのクロック数 / PRESCALER_DIV)	$(100_000_000 / \text{PRESCALER_DIV}) * 3$ (クロック周波数 100 MHz で動作時 3sec)
TX_PERIOD_SEDP_SUB_AN_COUNT	SEDP Subscription WriterのACKNACKパケット送信間隔 (ROS2rapperのクロック数 / PRESCALER_DIV)	$(100_000_000 / \text{PRESCALER_DIV}) * 3$ (クロック周波数 100 MHz で動作時 3sec)
TX_PERIOD_APP_WR_COUNT	アプリケーション・データ(Application WriterのDATAパケット)送信間隔 (ROS2rapperのクロック数 / PRESCALER_DIV)	$(100_000_000 / \text{PRESCALER_DIV}) * 3$ (クロック周波数 100 MHz で動作時 3sec)

Table A-3 定数一覧

定数名	説明	デフォルト値	定義されている場所 (*1)	変更時の注意点
ROS2_MAX_NODE_NAME_LEN (MAX_NODE_NAME_LEN) *2	ROS2ノード名の最大長	32	include/ros2_config.vh (hls/ros2.hpp)	CyberWorkBench 使用時には、RTL の修正が必要
ROS2_MAX_TOPIC_NAME_LEN (MAX_TOPIC_NAME_LEN) *2	ROS2トピック名の最大長	32	include/ros2_config.vh (hls/ros2.hpp)	CyberWorkBench 使用時には、RTL の修正が必要
ROS2_MAX_TOPIC_TYPE_NAME_LEN (MAX_TOPIC_TYPE_NAME_LEN) *2	ROS2トピック型名の最大長	64	include/ros2_config.vh (hls/ros2.hpp)	CyberWorkBench 使用時には、RTL の修正が必要
ROS2_MAX_APP_DATA_LEN (MAX_APP_DATA_LEN) *2	ROS2メッセージ・データの最大長	1024	include/ros2_config.vh (hls/ros2.hpp)	併せて hls/ros2.hpp 内の app_data_len_t を修正すること。
ROS2_APP_DATA_LEN_WIDTH (app_data_len_t) *2	ros2_pub_app_data_len_{0,1,2,3} のビット幅	$\$clog2(\text{'ROS2_MAX_APP_DATA_LEN'} + 1)$	include/ros2_config.vh (hls/ros2.hpp)	

定数名	説明	デフォルト値	定義されている場所 (*1)	変更時の注意点
ROS2_PUB_DATA_RAM (PUB_DATA_RAM) *2	送信アプリケーション・データのポートをメモリ・ポートにする。		include/ros2_config .vh (hls/ros2.hpp)	代わりに ROS2_PUB_DATA_FF (PUB_DATA_FF) を 定義すると、送信アプリケーション・データが レジスタ入力 (トピック 名等と同様の形式) になる。
ROS2_SEDP_READER_TBL_RAM (SEDP_READER_TBL_RAM) *2	Participant Table を ROS2rapper 外部のメモリ に保持する。		include/ros2_config .vh (hls/ros2.hpp)	代わりに ROS2_SEDP_READER_TBL_FF (SEDP_READER_TBL_FF) を定義すると、 Participant Table が ROS2rapper 内部の FF で実装される。
ROS2_APP_READER_TBL_RAM (APP_READER_TBL_RAM) *2	Endpoint Table を ROS2rapper 外部のメモリ に保持する。		include/ros2_config .vh (hls/ros2.hpp)	代わりに ROS2_APP_READER_TBL_FF (APP_READER_TBL_FF) を定義すると、 Endpoint Table が ROS2rapper 内部の FF で実装される。
ROS2_PUB_TOPICS_MAX (PUB_TOPICS_MAX) *2	パブリッシュ 可能な最大トピック数	4	include/ros2_config .vh (hls/endpoint.hpp)	HLS C++・RTL の修正が必要
ROS2_SUB_TOPICS_MAX (SUB_TOPICS_MAX) *2	サブスライブ可能な最大トピック数	4	include/ros2_config .vh (hls/endpoint.hpp)	HLS C++・RTL の修正が必要
ROS2_RTPS_DATA_WIDTH	HLS モジュール ros2_main のAXIS 入力 ポート in の TDATA の ビット幅	Vitis HLS 使用 時: 216 CyberWorkBench 使用時: 211	include/ros2_config .vh	高位合成結果に併せて修正すること。
ROS2_MESSAGE_METADATA_DATA_WIDTH	HLS モジュール ros2_main のAXIS 出力 ポート out の TDATA の ビット幅	Vitis HLS 使用 時: 384 CyberWorkBench 使用時: 374	include/ros2_config .vh	高位合成結果に併せて修正すること。
ROS2_SEDP_READER_MAX (SEDP_READER_MAX) *2	Participant Table に保持 できるノード	128	include/ros2_config .vh (hls/endpoint.hpp)	HLS C++・RTL の修正が必要

定数名	説明	デフォルト値	定義されている場所 (*1)	変更時の注意点
	情報の数			
ROS2_APP_READER_MAX (APP_READER_MAX) *2	Endpoint Table に保持 できるトピック 情報の数	128	include/ros2_config .vh (hls/endpoint.hpp)	HLS C++・RTL の修 正が必要
TARGET_PARTICIPANT_ID	ROS2ノード のParticipant ID	1	hls/common.hpp	
PAYLOADSMEM_DEPTH	IP受信ペイ ロード・メモリ のワード数	2960	include/ros2_config .vh	
PAYLOADSMEM_AWIDTH	IP受信ペイ ロード・メモリ のアドレス幅	$\$clog2(\text{PAYLOADSMEM_DEPTH})$	include/ros2_config .vh	
MAX_PENDINGS	同時にIPフラ グメント組み 立て待ちを行 えるIPデータ グラムの個数	1	hls/ip.hpp	
IP_MAX_PAYLOAD_LEN	最大IP受信 ペイロード・メ モリサイズ	1480	hls/ip.hpp	1480=MTU(1500) - IPヘッダサイズ(20)
MAX_IP_FRAGMENTS	最大IP受信 ペイロードフラ グメント数	2	hls/ip.hpp	
MAC_TX_FIFO_DEPTH	MAC内送信 FIFOの深さ	2048	ros2rapper/src/ethe rinclude/ros2_ether _config.vh	2のべきでなければな らない
MAC_RX_FIFO_DEPTH	MAC内受信 FIFOの深さ	2048	ros2rapper/src/ethe rinclude/ros2_ether _config.vh	2のべきでなければな らない
EXT_TX_FIFO_DEPTH	送信FIFOの 深さ	16	ros2rapper/src/ethe rinclude/ros2_ether _config.vh	2のべきでなければな らない
EXT_RX_FIFO_DEPTH	受信FIFOの 深さ	2048	ros2rapper/src/ethe rinclude/ros2_ether _config.vh	2のべきでなければな らない

定数名	説明	デフォルト値	定義されている場所 (*1)	変更時の注意点
UDP_PAYLOAD_FIFO_DEPTH	UDP 送信モジュール内のペイロード FIFO の深さ	2048	ros2rapper/src/etherinclude/ros2_ether_config.vh	2のべきでなければならない
UDP_HEADER_FIFO_DEPTH	UDP 送信モジュール内のヘッダ情報の FIFO の深さ	2	ros2rapper/src/etherinclude/ros2_ether_config.vh	2のべきでなければならない
ARP_CACHE_ADDR_WIDTH	ARPキャッシュのアドレス幅	7	ros2rapper/src/etherinclude/ros2_ether_config.vh	
ROS2_MAX_RAW_ETH_TX_DATA_LEN	Ethernet フレーム送信データ・メモリの容量 [byte]	1600	ros2rapper/src/etherinclude/ros2_ether_config.vh	
ROS2_MAX_RAW_ETH_RX_DATA_LEN	Ethernet フレーム受信データ・メモリの容量 [byte]	1600	ros2rapper/src/etherinclude/ros2_ether_config.vh	

注)

*1 パスは、トップ・ディレクトリからの記述の無いものは、ros2rapper/src/ros2rapper/ 以下を記述

*2 これらの定数は、VerilogとHLS C++のヘッダに存在し、定数名が異なる。括弧内に記述したものはHLS C++のヘッダとその所在である。これらを同時に変更する必要がある。

7.2. ディレクトリ構成

```
ros2rapper
|
|- src
|   |- ros2rapper          : ROS2rapper
|   |   |- hls             : HLS C++記述
|   |   |- include         : Verilog ヘッダファイル
|   |   |- rtl             : Verilog RTL
|   |   |- test            : C-Sim用テストベンチ
|   |
|   |- ether               : Ethernet層
|   |   |- include         : Verilog ヘッダファイル
|   |   |- rtl             : Verilog RTL
|   |   |- lib             : Verilog RTL
|   |   |- verilog-ethernet : Ethernet MACのVerilog RTL
|   |                       verilog-ethernet (参考文献 [5]) を一部改変して使用
|   |   |- tb              : RTL Simulation 用テストベンチ
|   |
|   |- examples
|   |   |- topic-pub-sub    : トピックのパブリッシュ/サブスクライブの実装例
|   |   |- raw-eth-tx-rx    : Ethernet フレーム送受信の実装例
|   |   |- multi-topic-publication : 複数のトピックをパブリッシュする実装例
|   |   |- multi-topic-subscription: 複数のトピックをサブスクライブする実装例
|   |   |- test-1KB-pub-sub : ROS2, Ethernet フレーム送受信の最大長での使用例
|   |   |- topic-pub-sub-au25p : Artix Ultrascale+ AU25P での実装例
|   |
|   |- formal : SymbiYosysを使用したフォーマル検証用のスクリプト
|   |
|   |- README.md
```

7.3. RTPS通信で使用するUDPポート番号 一覧

RTPS通信では、下記のUDPポート番号を受信に使用する。

- SPDP_WELL_KNOWN_MULTICAST_PORT
 - $7400 + \text{DOMAIN_ID} * 250$
- METATRAFFIC_UNICAST_LOCATOR
 - $7400 + \text{DOMAIN_ID} * 250 + 10 + 2 * \text{PARTICIPANT_ID}$
- DEFAULT_UNICAST_LOCATOR
 - $7400 + \text{DOMAIN_ID} * 250 + 11 + 2 * \text{PARTICIPANT_ID}$

式中のDOMAIN_IDは、下記の値である。

(ros2_port_num_seedポートの値) - 7400 / 250

8. 変更履歴

ver 2.0

1. パブリッシュ、サブスクライブ可能なトピック数を、それぞれ 1 つから 4 つに増やした。
 - トピックの enable 信号、トピック名等の設定ポート、アプリケーション・データのポート数や bit 幅を増加させた。(§3.2, 3.5, 3.6, 3.7, 4.1, 4.2, 4.3, 7.1)
2. Ethernet フレーム送受信機能を追加した。(§1.3, 5.1, 5.2, 7.1)
3. UDP データグラム送受信機能を削除した。
4. パブリッシュするアプリケーション・データのポートを、メモリ・ポートに変更した。(§2, 3.6, 4.2, 7.1)
5. 通信相手のトピック情報・ノード情報のテーブルを、ROS2rapper 内部の FF から ROS2rapper 外部のメモリに変更した。(§2, 3.11, 3.12, 7.1)
6. 開発環境の情報を追加した。(§3.1)
7. RTL 合成時のマクロについて情報を追加した (§3.1)
8. パケットの送信間隔の設定方法を変更・追加した。(§3.4, 7.1)
 - 関連するパラメータ、ポートを追加した。
9. 設定項目に
 - RTPS のベンダー ID
 - Participant Lease Durationを追加した。(§3.5, 7.1)
10. アプリケーション・データ・メモリのハンドシェイクのプロトコルを変更した。(§4.2, 4.3, 7.1)
11. RTPS でアプリケーション・データを受信した際に、ユーザ・ロジックに受信を通知する方法を変更した。(§4.3, 7.1)
12. 対向ノードの離脱を検知する機能を実装した。
13. 同時に通信可能な通信相手の数 (ノード単位、トピック単位) を 4 から 128 に増加させた。(§7.1)
14. アプリケーション・データの最大長を 64 bytes から 1024 bytes に増加させた。(§7.1)
15. アーキテクチャの項を更新した。(§2)
16. 実装例 (examples/)・テストベンチマークの情報を更新した。(§2, 7.2)
17. その他の修正
 - アプリケーション・データ・メモリ (受信) のポートの項に、ROS2rapper が read するポートが記載されていたが、これは存在しないポートだった。そのため、この記載を削除した。(§3.7)
 - 付録 7.1 の ros2_fragment_expiration の項目の説明を修正した。(§7.1)

ver 1.0

- 公開